CS 614 - Technical Report: Word2Vec with CBOW

Gary Pham - gp492

February 2026

1 Description of the Dataset You Created

Work your way through the Jupyter notebook. This technical report includes:

1.1 Dataset Statistics

The dataset consists of **28 sentences** containing **66 unique words** after preprocessing. The preprocessing step involved simple regex tokenization with lowercasing, using the pattern `[a-z']+` to extract alphabetic tokens and drop punctuation marks.

1.2 Example Sentences

The following are representative examples from the dataset:

- What is the best time to call you tomorrow?
- What is the best hour to call you tomorrow?
- What is the best time to contact you tomorrow?
- What is the best hour to contact you tomorrow?
- Can we schedule a meeting for tomorrow morning?

The dataset was intentionally designed with overlapping contexts and semantic relationships. For instance, sentences contain synonyms (e.g., `time`/`hour`, `call`/`contact`/`phone`), related concepts (e.g., `meeting`/`appointment`), and other semantic pairs (e.g., `car`/`automobile`, `quick`/`fast`). This design allows the embedding space to learn useful similarity structure from a relatively small corpus.

2 Hyperparameter Design Choices

2.1 Window Size

The context window size was set to **5**, meaning that for each target word, we consider 2 words before and 2 words after it as context. This is an odd integer window size, which ensures a balanced context on both sides of the target word. The window size of 5 is consistent with common practices in word2vec implementations and provides a good balance between capturing local context and computational efficiency.

2.2 Embedding Dimension

The embedding dimension was set to **300**, following the original word2vec paper which demonstrated that 300-dimensional embeddings provide an optimal trade-off between embedding quality and computational cost. Each word in the vocabulary is represented as a 300-dimensional vector.

2.3 Number of Training Epochs

The model was trained for **500 epochs**. This number was chosen to ensure sufficient training for convergence on the small synthetic dataset. The training loss was monitored throughout training to verify convergence.

2.4 Loss Function, Optimizer, and Related Hyperparameters

- **Loss Function**: `CrossEntropyLoss` from PyTorch. This is appropriate for the multi-class classification task where we predict the center word from context words. PyTorch's `CrossEntropyLoss` expects raw logits (unnormalized scores) and applies softmax internally.
- **Optimizer**: `Adam` optimizer with an initial learning rate of **0.025**. Following the original word2vec paper recommendations, a learning rate scheduler was used to linearly decay the learning rate from 0.025 to 0 over the course of training. This gradual decay helps stabilize training and improve convergence.
- **Batch Size**: **32**. This batch size provides a good balance between training stability and computational efficiency for the dataset size.

2.5 Additional Design Choices

- **Model Architecture**: The CBOW (Continuous Bag of Words) model was implemented using PyTorch's `nn.Embedding` layer followed by a linear classifier. The model architecture consists of:
  - An `Embedding` layer that maps word indices to 300-dimensional vectors
  - A mean pooling operation that averages the context word embeddings
  - A `Linear` layer that maps the averaged embedding to vocabulary-sized logits
- **Embedding Regularization**: The embedding layer uses `max_norm=1` to restrict embedding vector norms. This acts as a regularization technique and prevents embedding weights from growing uncontrollably during training. It has been shown to improve similarity relationships between related words.
- **Tokenization**: Simple regex-based tokenization (`re.findall(r"[a-z']+", ...)`) was used to lowercase all text and extract only alphabetic tokens, dropping punctuation marks. This is intentionally simple since the dataset is small and synthetic.
- **Implementation Details**: The implementation follows the approach described in the Towards Data Science article on word2vec with PyTorch, using `nn.Embedding` rather than one-hot encoding with linear layers for better efficiency.

3 Results

3.1 Training Plot

Figure 1 shows the training loss over 500 epochs. The loss decreases from an initial value of approximately 3.86 to a final value of 0.0681, demonstrating successful convergence. The plot includes annotations marking the initial and final loss values.

Figure 1: CBOW training loss over 500 epochs. The loss decreases from 3.86 to 0.0681, showing successful convergence.

3.2 Model Performance Metrics

The trained model achieved the following performance metrics:

- **Final Training Loss**: 0.0681
- **Top-1 Accuracy**: 95.40% (exact match accuracy)
- **Top-3 Accuracy**: 100.00% (correct word in top 3 predictions)
- **Top-5 Accuracy**: 100.00% (correct word in top 5 predictions)
- **Perplexity**: 1.07

The high accuracy metrics indicate that the model successfully learned to predict center words from their context. The perplexity of 1.07 is very low, indicating high confidence in predictions, which is expected given the small, controlled vocabulary and synthetic nature of the dataset.

3.3 Results of Similar Word Tests

To evaluate the quality of the learned word embeddings, cosine similarity was computed between word vectors. Table 1 shows the top-5 most similar words for a set of test words, along with their cosine similarity scores.

Table 1: Top-5 most similar words (by cosine similarity) for selected test words.

Word	Top Similar Words (cosine similarity)
time	hour (0.817), dog (0.357), for (0.332), you (0.231), up (0.220)
hour	time (0.817), you (0.460), is (0.352), works (0.309), dog (0.283)
call	contact (0.777), best (0.719), good (0.371), meeting (0.271), afternoon (0.205)
contact	call (0.777), best (0.553), good (0.342), have (0.269), you (0.209)
meeting	he (0.622), morning (0.590), afternoon (0.524), dog (0.434), arrange (0.409)
appointment	purchased (0.554), automobile (0.554), she (0.554), yesterday (0.554), set (0.363)
car	bought (1.000), last (0.471), arrange (0.287), meeting (0.253), we (0.237)
automobile	purchased (1.000), yesterday (1.000), she (1.000), plan (0.572), appointment (0.554)
quick	fast (0.830), brown (0.670), leaps (0.484), the (0.221), over (0.206)
fast	quick (0.830), brown (0.664), jumps (0.469), leaps (0.302), over (0.207)

3.3.1 Analysis of Similarity Results

The similarity results demonstrate that the model successfully learned semantic relationships:

- **Synonym pairs** show high similarity: time/hour (0.817), call/contact (0.777), and quick/fast (0.830) are correctly identified as the most similar words to each other.
- **Related concepts** are captured: car is most similar to bought (1.000), and automobile is most similar to purchased (1.000), reflecting the co-occurrence patterns in the training sentences.
- **Contextual relationships** are learned: meeting is associated with temporal words like morning and afternoon, and quick/fast are associated with brown, reflecting their co-occurrence in the "quick brown fox" sentences.
- Some results reflect the small dataset size: words like dog appear in similarity lists due to frequent co-occurrence in the training sentences, even when not semantically related to the query word.

Overall, the model demonstrates that CBOW successfully learns meaningful word embeddings that capture semantic and contextual relationships, even from a small synthetic dataset.